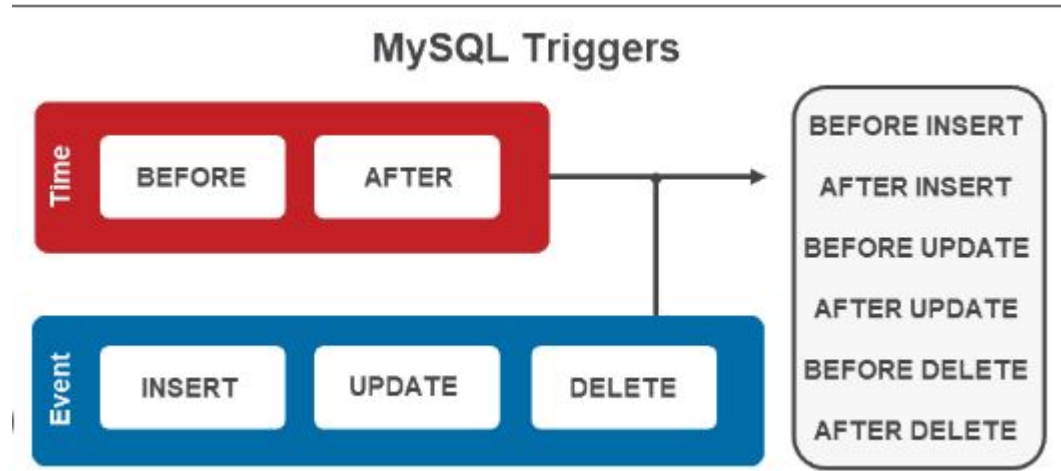


TRIGGER

¿Que es un Trigger?



Definimos como Trigger a una **aplicación o programa almacenado en el servidor** (de base de datos) creado para ejecutarse (dispararse) de **forma automática**, cuando uno o más eventos específicos ocurren en la Base de Datos.



Un **trigger** o **disparador** es una regla que se asocia a una tabla. Mediante esta regla, se ejecutan una serie de instrucciones cuando se producen ciertos eventos sobre una tabla. Los eventos son:
INSERT, UPDATE o DELETE.

Triggers



Los **Triggers** nacieron integrados a las bases de datos para funcionar de la misma forma en la cual muchos lenguajes de programación activan la detección de eventos específicos cuando el programa se ejecuta.

Pero un **Trigger** no funciona de forma mágica. Nosotros somos quienes debemos definirlo y escribir su lógica.

FUNCIONALIDAD DE LOS TRIGGERS

The screenshot displays a complex database schema diagram. It features several tables represented as boxes, interconnected by lines indicating relationships. Key tables visible include:

- ANA_OBJECT**: Contains fields like OBJ_ID, OBJ_CREATION_DATE_TIME, and OBJ_CREATOR_FK.
- ANA_RELATIONSHIP**: Contains fields like REL_OID (FK), REL_RELATIONSHIP, REL_CHILD_FK (FK), REL_PARENT_FK, and REL_SEQUENCE.
- ANA_SOURCE**: Contains fields like SRC_OID (FK), SRC_NAME, SRC_AUTHOR, and SRC.
- ANA_ATTRIBUTION**: Contains fields like ATL_OID (FK), ATL_OBJECT_FK (FK), ATL_SOURCE_FK (FK), ATL_EVIDENCE_FK (FK), and ATL_COMMENTS.

The diagram uses various symbols (diamonds, lines) to denote different types of relationships and constraints between these tables.

Estas tablas **funcionan como complemento dentro de una bb.dd.**, recopilando información adicional que no es importante dentro de las tablas principales

Triggers - Logs

Tablas de **Auditoría, Log, Bitácora**, son algunos nombres con los cuales se definen a estas tablas secundarias, que se ocupan de almacenar información no importante para el negocio en sí, pero **clave para el departamento de IT y/o de Seguridad Informática**.

LOGIN_HISTORY		
id	user_id	login_datetime
1	2	2021-05-21 11:00:23
2	1	2021-05-21 11:10:23
3	3	2021-05-22 11:10:23
4	2	2021-05-22 15:10:23
5	1	2021-05-22 16:43:23

Triggers - Logs

Por ejemplo una tabla **Productos**, almacena información del mismo como: idProducto, descripción, precio de costo, precio de venta, idMarca, entre otros.

Podemos registrar en una tabla de **auditoría** paralela, quién lo creó, fecha de creación, quién modificó su precio de venta o de costo, cuándo, y quién lo eliminó de la lista de Productos.



Triggers - Logs



Todo este registro o bitácora de cambios, lo podemos realizar activando uno o más **Triggers** que monitoreen todos estos pasos y cambios sobre uno o más registros.

Las tablas de **LOGs** o **Bitácoras** no suelen tener relación alguna con las entidades que monitorean y, muchas veces, almacenan información general de diferentes entidades.

TIMING BEFORE - AFTER

TIMING DE LOS TRIGGERS



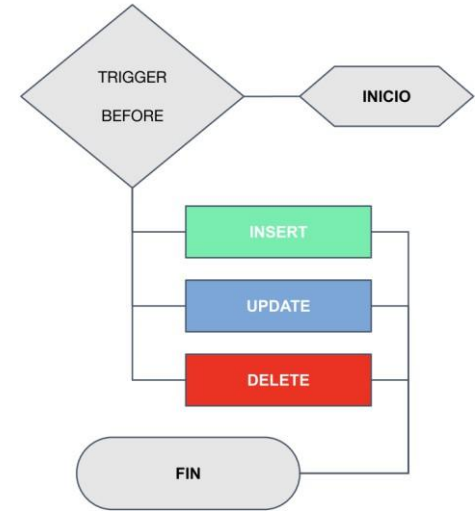
El uso de **Trigger** se puede establecer en dos momentos diferentes de cuando se realiza una operación del tipo **UPDATE**, **DELETE**, o **INSERT**.

- Ese momento puede ser antes (**BEFORE**) de que ocurra la operación o después (**AFTER**) de que ocurra la misma

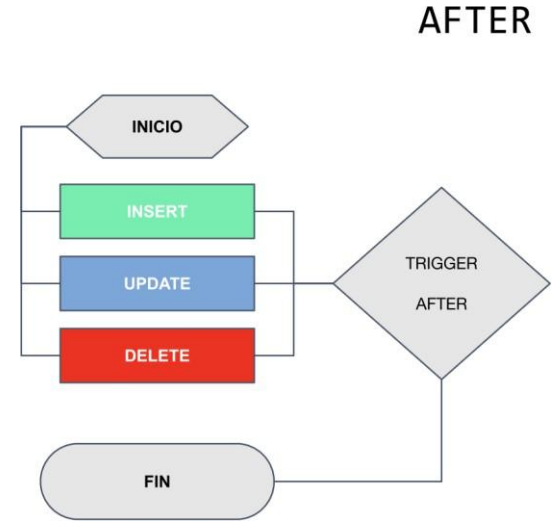
Cuando el usuario envía una operación del tipo **INSERT**, **UPDATE** o **DELETE** sobre una tabla, y esta tiene activo el **Trigger** que detecta la operación, se disparará la acción **BEFORE**, la cual permitirá por ejemplo registrar en una tabla de auditoría que se realizará la operación x sobre la tabla y.



BEFORE



Cuando el usuario envía una operación del tipo **INSERT**, **UPDATE** o **DELETE** sobre una tabla, y esta tiene activo el **Trigger** que detecta la operación, se disparará la acción **AFTER**, la cual registrará en una tabla de auditoría que se realizará la operación x sobre la tabla y.



VALOR NEW Y OLD

Instrucciones a emplear dentro de los triggers

Habitualmente lo que se introduce dentro de los disparadores son instrucciones condicionales, en otras palabras secuencias que se ejecutarán si se da un caso concreto, aunque no es obligatorio. Para poder manejar estas instrucciones condicionales tenemos las variables **OLD** y **NEW**.



VARIABLE NEW

NEW almacena el valor que aporta la consulta a la base de datos. Con esta variable podemos acceder a los datos introducidos.

Con **NEW.nombre_columna** se almacenará la información con el nuevo valor que tendrá ese registro modificado (desde un **UPDATE** o **INSERT**) en la tabla.

Los trigger relacionados con DELETE no tendrán disponible la variable NEW.

VARIABLE OLD

OLD a diferencia de **NEW**, almacena el valor de las columnas que van a ser borradas o actualizadas.

Al igual que pasa con NEW, **OLD no está disponible en todas las instrucciones, más concretamente el valor no se puede recuperar cuando la instrucción es un INSERT.**

RESUMEN

A TENER EN CUENTA:

Cuando hacemos un INSERT las referencias NEW son válidas

Cuando hacemos un UPDATE las referencias NEW y OLD son válidas

Cuando hacemos un DELETE las referencias OLD son válidas

SQL

FOREACH ROW

FOR EACH ROW

Un **Trigger** es un evento que se dispara una vez por cada sentencia **INSERT** que se realice sobre la tabla asociada. Pero para aquellos casos donde se inserten registros de forma masiva, necesitamos incluir la cláusula **FOR EACH ROW**.

De esta forma nos aseguramos que el **Trigger** se disparará una vez por cada nuevo registro agregado de forma masiva.

EJEMPLO 1

COMPROBACIÓN DE DATOS

Imaginemos que tenemos una tabla clientes en la que almacenamos su edad. Para evitar problemas en el tratamiento de los datos, queremos evitar que se introduzcan edades negativas. Podríamos hacer algo parecido a:

SQL

```
DELIMITER //  
  
CREATE TRIGGER comprobar_edad  
BEFORE INSERT ON cliente  
FOR EACH ROW  
BEGIN  
    IF NEW.edad < 0 THEN  
        SET NEW.edad = 0;  
    END IF;  
END//
```

Con este sencillo **trigger** evitaremos que en nuestra base de datos se puedan guardar edades menores a cero, de manera que podríamos omitir esta comprobación en desarrollos que hiciésemos con estos datos.

EJEMPLO 2

Realización de un histórico de productos

Imaginemos que tenemos una tienda online.

Quizás nos interese tener una tabla en la que guardamos todos los productos que hemos tenido en nuestro catálogo en algún momento.

Podríamos introducir los datos dentro del histórico siempre que introdujésemos un nuevo producto. Sin embargo, **¿No sería más sencillo automatizar un histórico de los productos?**

Gracias a los **trigger** podremos realizarlo sin tener que ejecutar dos instrucciones separadas.



Realización de un histórico de productos

SQL

```
DELIMITER //  
  
CREATE TRIGGER trigger_producto_historico  
AFTER INSERT ON producto  
  
FOR EACH ROW  
  
BEGIN  
    INSERT INTO producto_historico (id,descripcion,precio,fecha_hora) VALUES (NEW.id,  
NEW.nombre, NEW.precio, NOW());  
  
END//
```

De esta forma, cada vez que incluyamos un **producto nuevo**, este también se añadirá a la tabla **producto_historico**, y aunque posteriormente borremos el artículo de la tabla producto, **este seguirá existiendo en producto_historico**.

CONCLUSIÓN

A través de los ejemplos que hemos visto, hemos descubierto que gracias a los **disparadores** o **triggers** podemos **automatizar** acciones fácilmente con MySQL y mantener nuestra base de datos actualizada.

